Naruto Shinobi API

Backend Architecture & Implementation Summary

PHASE 1

Project Initialization & Docker Setup

The foundation was built using .NET 10 (Preview/Current) and SQL Server 2022 running inside a Docker container to ensure environment consistency.

Docker Compose Configuration

```
services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      SA_PASSWORD: YourPassword123!
      ACCEPT_EULA: Y
    ports:
      - "1433:1433"
```

Key Achievement: Overcame the 'sa' login failure and port conflicts by standardizing on port 1433 and verifying container health with `docker ps`.

PHASE 2

Data Modeling & Relational Logic

We evolved the data from a single "flat" table to a professional relational schema involving three main entities.

- **Villages:** The parent entity (One-to-Many with Shinobis).

- **Shinobis:** The central entity, including `PhotoUrl` for image support.
- **Jutsus:** The skill entity (Many-to-Many with Shinobis).

The Many-to-Many Evolution

Instead of a simple foreign key, we implemented `List<Jutsu>` and `List<Shinobi>` in both classes, allowing EF Core to generate the **Join Table** automatically.

PHASE 3

Entity Framework Core Migrations

Managed the database schema versioning through the .NET CLI.

```
# Creating the schema
dotnet ef migrations add RelationshipSetup
dotnet ef migrations add ManyToManyJutsu

# Applying to SQL Server
dotnet ef database update
```

Fixing the "Invalid Object Name" error: We resolved this by ensuring `database update` was run while the specific Docker container was active.

PHASE 4

API Endpoints & Middleware

Built using Minimal APIs for performance and clarity.

- **CORS Policy:** Enabled `AllowAngular` for `localhost:4200`.

- **JSON Circular References:** Configured `ReferenceHandler.IgnoreCycles` to prevent infinite loops when returning nested relationships (Village -> Shinobi -> Village).
- **Relationship Endpoints:** Created a specific POST route to link existing Shinobis to existing Jutsus via IDs.